Gradient descent and Neural Networks
October 12th, 2026

These slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge.

Announcements

- HW #2 is due Friday October 16, 11.59pm uploaded to Gradescope. **Note that even 1 minute late is considered a day late**

- HW#1 late date is tonight (2 day late submission). **Hard deadline as solutions will be posted tomorrow.**

- **Please read the rules handout carefully including for late HWs etc.**

- To submit your Jupyter Notebook, print the notebook to a pdf with your solutions and plots filled in. The graders will grade your work from these notebooks. In the future, you will also have to print out code from any .py files that were modified.

- Discussion sessions held on Fridays 11am, will be recorded and live zoom link also is available.

- For midterm **CAE approved accommodation students have to contact CAE office to proctor exam** as we cannot do this ourselves. Please do this pro-actively.

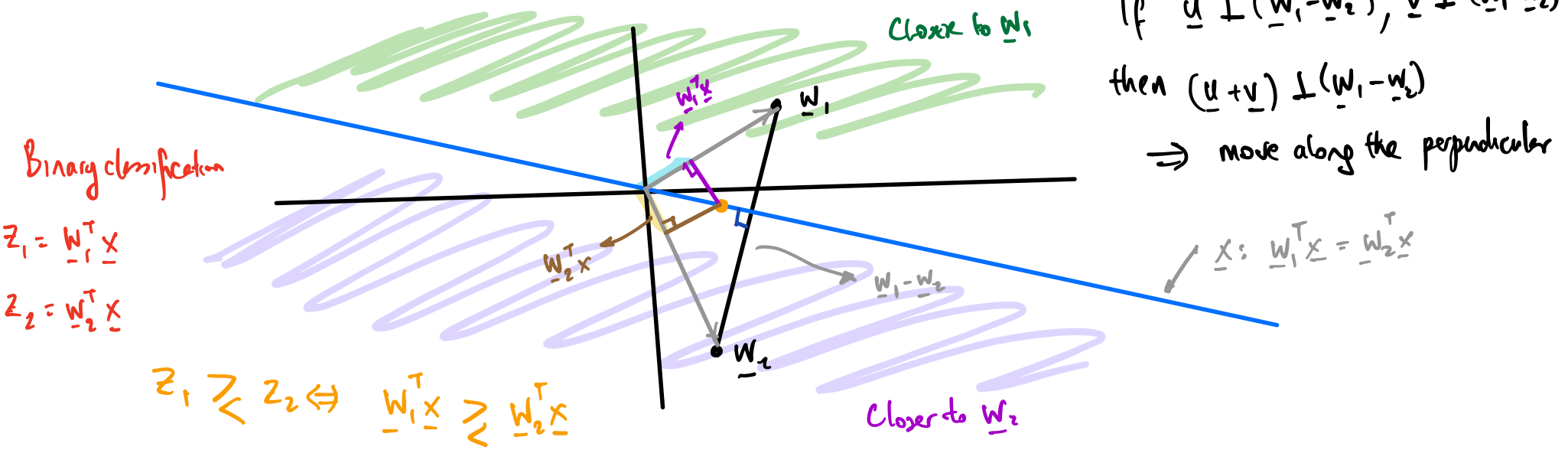
Review of previous lecture

- **Linear classifiers:**

$$\mathbf{W} = \begin{bmatrix} \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_c^T \end{bmatrix}, \quad \mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

"score" ← $\mathbf{z}$, $\mathbf{W}$, $\mathbf{x}$ ← input

, find the label by $k = \arg\max_j \mathbf{z}_j$ where $\mathbf{z}_j$ is j-th component of $\mathbf{z}$, and the estimated label is $\hat{y} = y_k$.



- **Question:** how to find appropriate $\mathbf{W}$: need a optimization criterion and method to solve for the criterion.

Review of previous lecture (Contd.)

- **Turning scores into probability: softmax function:** $\text{softmax}_i(\mathbf{x}) = \dfrac{e^{a_i(\mathbf{x})}}{\sum_{j=1}^{c} e^{a_j(\mathbf{x})}}$ where

$a_i(\mathbf{x}) = \mathbf{w}_i^\top \mathbf{x} + b_i$ and we interpret $\text{Prob}(y^{(i)} = j \,|\, \mathbf{x}^{(i)}, \theta) = \text{softmax}_j(\mathbf{x}^{(i)})$

$\prod_{i=1}^{N} p(x^{(i)}, y^{(i)}) = p(x^{(1)}, y^{(1)}) \cdot p(x^{(2)}, y^{(2)}) \cdots p(x^{(N)}, y^{(N)})$

- **Optimization criterion:** Maximum likelihood estimation

$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \mathscr{L}(\boldsymbol{\theta})$ where $\mathscr{L}(\boldsymbol{\theta}) = \mathbb{P}(\mathscr{D} \,|\, \boldsymbol{\theta})$, and $\mathscr{D} = \{(\boldsymbol{x}^{(1)}, y^{(1)}), \ldots, \boldsymbol{x}^{(N)}, y^{(N)})\}$ is generated i.i.d. according to $p(\boldsymbol{x}, y)$

$p(\mathscr{D}|\theta) = \prod_{i=1}^{N} p(x^{(i)}, y^{(i)})$

$\log p(\mathscr{D}|\theta) = \sum_{i=1}^{N} \log p(x^{(i)}, y^{(i)})$

- Writing loss function in terms of softmax: equivalent forms

Cross entropy form: $J(\boldsymbol{\theta}) = -\mathscr{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{m} \left(\sum_{j=1}^{c} z_j^{(i)} \log p_j^{(i)} \right)$

$z_j^{(i)} = \begin{cases} 1 & \text{if } j = y^{(i)} \\ 0 & \text{else} \end{cases}$ one hot encoding

$p(y^{(i)} = j \,|\, x^{(i)}, \theta) = \text{softmax}_j(x^{(i)})$

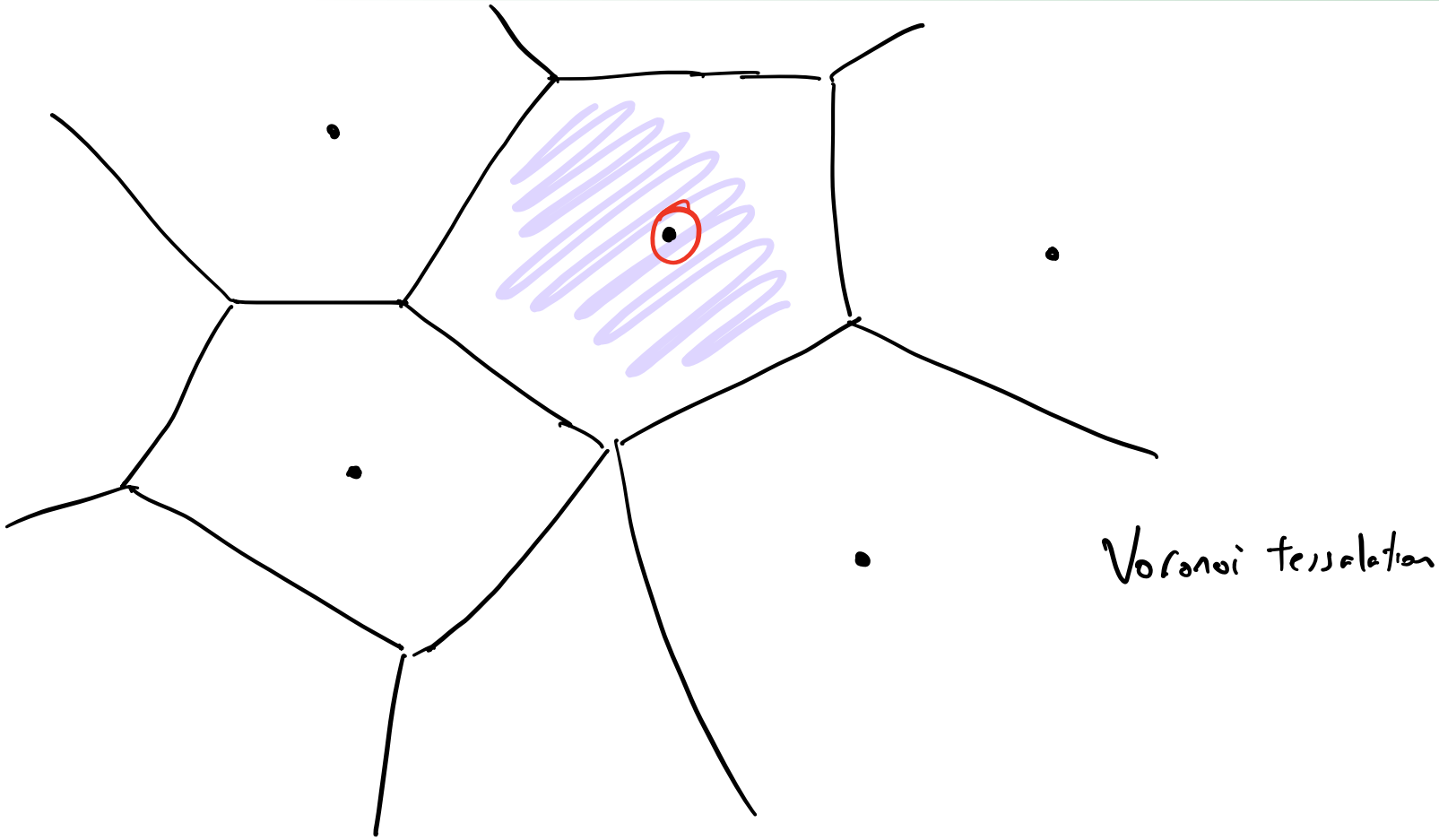
$\theta = \{W, b\}$

Softmax form: $J(\boldsymbol{\theta}) = -\log \mathscr{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{m} \left(\log \sum_{j=1}^{c} \left\{ e^{a_j(\mathbf{x}^{(i)})} \right\} - a_{y^{(i)}}(\mathbf{x}^{(i)}) \right)$

$\text{softmax}_{y^{(i)}}(x^{(i)})$

$z = W^\top x + b$, $z_j^{(i)} = w_j^\top x^{(i)} + b_j$ (want to find)

Review of previous lecture (Contd.)



$a_j = \begin{bmatrix} 100 \\ 20 \\ -1 \end{bmatrix}$

$\log\left[\sum e^{a_j(x)}\right] = \log\left[e^{100} + e^{20} + e^{-1}\right] \approx \log e^{100} = 100$

A few additional notes on the softmax classifier:

Softmax classifier: intuition

When optimizing likelihoods, we typically work with the "log likelihood." When applying the softmax, we interpret its output as the probability of a class.

$\mathcal{D} = \{ \underline{x}^{(i)}, y^{(i)} \}$

Correct label associated with $\underline{x}^{(i)}$ is $y^{(i)}$

$$\log \Pr(y = i|\mathbf{x}) = \log \text{softmax}_i(\mathbf{x})$$

$$\log \Pr(y = y^{(i)}|\mathbf{x}) = a_i(\mathbf{x}) - \log \sum_{j=1}^{c} \exp(a_j(\mathbf{x}))$$

$a_i(\mathbf{x})$: score of correct class; $a_{y^{(i)}}(\mathbf{x})$: correct label

for correct class ≈ 0

for any class this is $-ve$

When maximizing this, the term $a_i(\mathbf{x})$ is made larger, and the term $\log \sum_j \exp(a_j(\mathbf{x}))$ is made smaller. $a_{y^{(i)}}(\mathbf{x})$ is made larger, and the term by $\max_j a_j(\mathbf{x})$. (Why?)

We consider two scenarios:

- If $a_i(\mathbf{x})$ produces the largest score, then the log likelihood is approximately 0. $a_{y^{(i)}}(\mathbf{x})$ produces the largest score, then the log likelihood is approximately
- If $a_j(\mathbf{x})$ produces the largest score for $j \neq i$, then $a_i(\mathbf{x}) - a_j(\mathbf{x})$ is negative, and thus the log likelihood is $j \neq y^{(i)}$ ive. $a_{y^{(i)}}(\mathbf{x}) - a_j(\mathbf{x})$ is

A few additional notes on the softmax classifier:

Overflow of softmax

A potential problem when implementing a softmax classifier is overflow.

- If $a_i(\mathbf{x}) \gg 0$, then $e^{a_i(\mathbf{x})}$ may be very large, and numerically overflow and / or result to numerical impression.
- Thus, it is standard practice to normalize the softmax function as follows:

$a_i(\underline{x}) = \underline{w}_i^T \underline{x} + b_i$

if $\underline{w}_i^T \underline{x}$ is large

ie $\|\underline{w}_i\| \cdot \|\underline{x}\| \cdot \cos\theta$ is large

eg. if $a_i(\underline{x}) = 100$, e^{100} is large

$$\text{softmax}_i(\mathbf{x}) = \frac{e^{a_i(\mathbf{x})}}{\sum_{j=1}^{c} e^{a_j(\mathbf{x})}} = \frac{k e^{a_i(\mathbf{x})}}{k \sum_{j=1}^{c} e^{a_j(\mathbf{x})}} = \frac{e^{a_i(\mathbf{x}) + \log k}}{\sum_{j=1}^{c} e^{a_j(\mathbf{x}) + \log k}}$$

Could cause numerical overflow

Solution: Do normalized calculation

$\tilde{a}_i(\underline{x}) = a_i(\underline{x}) + \log k$

$-\max_j a_j(\underline{x})$

- A sensible choice of k is so that $\log k = -\max_i a_i(\mathbf{x})$, making the maximal argument of the exponent 0.

Softmax loss function

Softmax:

$$\arg\min_{\theta} \frac{1}{m} \sum_{i=1}^{m} \left(\log \sum_{j=1}^{c} e^{a_j(\mathbf{x})} - a_{y^{(i)}}(\mathbf{x}^{(i)}) \right)$$

$J(\underline{\theta})$

→ loss function

Solve this optimization problem

Parameters? → $\{ \underline{\underline{W}}, \underline{b} \}$

$\underline{\underline{W}} \in \mathbb{R}^{c \times n}$, $\underline{b} \in \mathbb{R}^{n}$

$\underline{x} \in \mathbb{R}^{n}$

GOAL: $\arg\min_{\theta} J(\underline{\theta})$

Big question: how do we find these parameters?

Finding the optimal weights through gradient descent

1st order Taylor series expansion $f(x+\epsilon) \approx f(x) + \epsilon f'(x)$

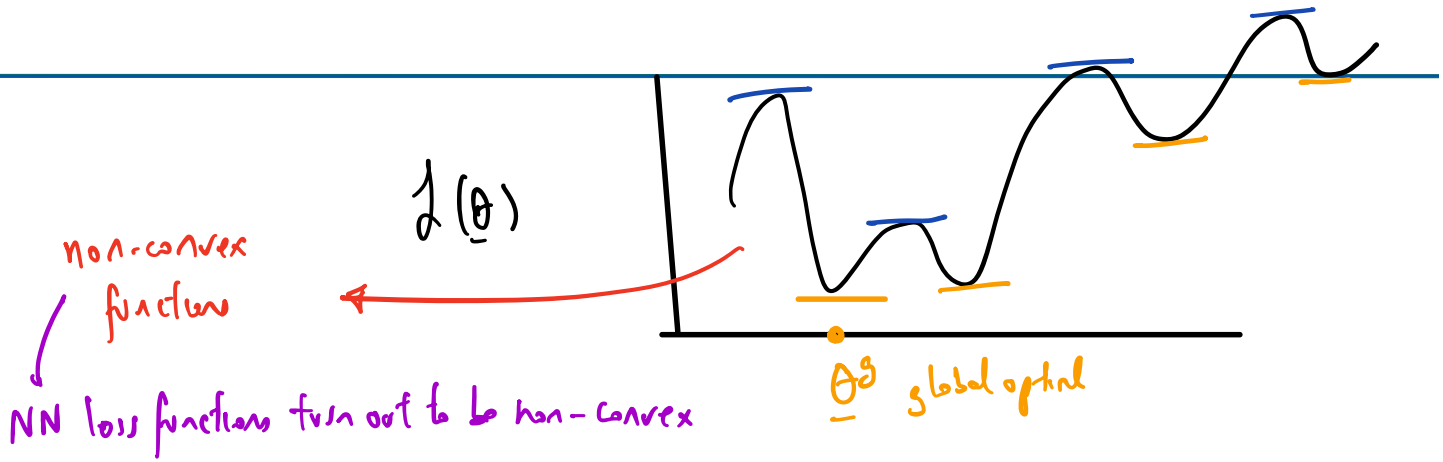
- Our goal in machine learning is to optimize an objective function, $f(x)$. (Without loss of generality, we'll consider minimizing $f(x)$. This is equivalent to maximizing $-f(x)$.)
- From basic calculus, we recall that the derivative of a function, $\frac{df(x)}{dx}$ tells us the slope of $f(x)$ at point x.
  - For small enough ϵ, $f(x+\epsilon) \approx f(x) + \epsilon f'(x)$.
  - This tells us how to reduce (or increase) $f(\cdot)$ for small enough steps.
  - Recall that when $f'(x) = 0$, we are at a stationary point or critical point. This may be a local or global minimum, a local or global maximum, or a saddle point of the function.
- In this class we will consider cases where we would like to maximize f w.r.t. vectors and matrices, e.g., $f(\mathbf{x})$ and $f(\mathbf{X})$.
- Further, often $f(\cdot)$ contains a nonlinearity or non-differentiable function. In these cases, we can't simply set $f'(\cdot) = 0$, because this does not admit a closed-form solution.
- However, we can iteratively approach an critical point via gradient descent.

$J(\underline{\theta})$ → translate $x \to \underline{\theta}$ for this part of lecture

$\underline{x} \to f(\underline{x})$ if I change the value to $\underline{x}+\underline{\epsilon}$ for small $\underline{\epsilon}$ how does $f(\underline{x}+\underline{\epsilon})$ compare to $f(\underline{x})$

APPROACH: To do so, we use the technique of gradient descent.

Finding the optimal weights through gradient descent



Terminology

- A **global minimum** is the point, $\mathbf{x}_g$, that achieves the absolute lowest value of $f(\mathbf{x})$. i.e., $f(\mathbf{x}) \geq f(\mathbf{x}_g)$ for all $\mathbf{x}$.
- A **local minimum** is a point, $\mathbf{x}_\ell$, that is a critical point of $f(\mathbf{x})$ and is lower than its neighboring points. However, $f(\mathbf{x}_\ell) > f(\mathbf{x}_g)$.
- Analogous definitions hold for the **global maximum** and **local maximum**.
- A **saddle point** are critical point of $f(\mathbf{x})$ that are not local maxima or minima. Concretely, neighboring points are both greater than and less than $f(\mathbf{x})$.

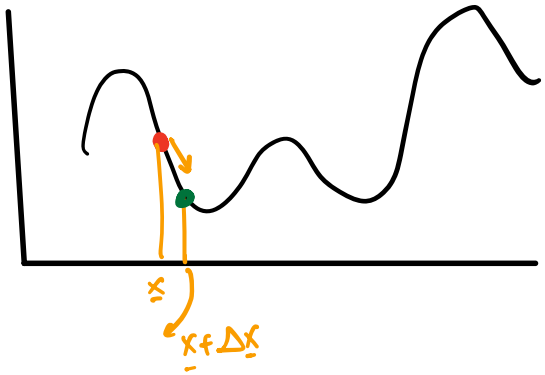
In high dimensions even though getting to global optimum might be difficult (or atleast not easy to guarantee), local minima are quite good empirically.

Finding the optimal weights through gradient descent

Solve $\arg\min_{\underline{\theta}} J(\underline{\theta})$

Gradient

Recall the gradient, $\nabla_{\mathbf{x}} f(\mathbf{x})$, is a vector whose ith element is the partial derivative of $f(\mathbf{x})$ w.r.t. x_i, the ith element of $\mathbf{x}$. Concretely, for $\mathbf{x} \in \mathbb{R}^n$,



$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial (x_n)} \end{bmatrix}$$

$\underline{x} + \Delta\underline{x}$ → want to choose $\Delta\underline{x}$ to make $f(\underline{x})$ smaller

- The gradient tells us how a small change in $\Delta\mathbf{x}$ affects $f(\mathbf{x})$ through

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \Delta\mathbf{x}^T \nabla_{\mathbf{x}} f(\mathbf{x})$$

1st order Taylor series

- The directional derivative of $f(\mathbf{x})$ in the direction of the unit vector $\mathbf{u}$ is given by:

$$\mathbf{u}^T \nabla_{\mathbf{x}} f(\mathbf{x})$$

$\|\underline{u}\| = 1$

- The directional derivative tells us the slope of f in the direction $\mathbf{u}$.

Finding the optimal weights through gradient descent

$f(\underline{x}+\Delta\underline{x}) - f(\underline{x}) \approx \Delta\underline{x}^T \nabla_{\underline{x}} f(\underline{x})$

Arriving at gradient descent

- To minimize $f(\mathbf{x})$, we want to find the direction in which $f(\mathbf{x})$ decreases the fastest. To do so, we find the direction $\mathbf{u}$ which minimizes the directional derivative.

$$\begin{aligned} \min_{\mathbf{u}, \|\mathbf{u}\|=1} \mathbf{u}^T \nabla_{\mathbf{x}} f(\mathbf{x}) &= \min_{\mathbf{u}, \|\mathbf{u}\|=1} \|\mathbf{u}\| \, \|\nabla_{\mathbf{x}} f(\mathbf{x})\| \cos\theta \\ &= \min_{\mathbf{u}} \nabla_{\mathbf{x}} f(\mathbf{x}) \cos(\theta) \end{aligned}$$

  where θ is the angle between the vectors $\mathbf{u}$ and $\nabla_{\mathbf{x}} f(\mathbf{x})$.
- This quantity is minimized for $\mathbf{u}$ pointing in the opposite direction of the gradient, so that $\cos(\theta) = -1$.
- Hence, we arrive at gradient descent. To update $\mathbf{x}$ so as to minimize $f(\mathbf{x})$, we repeatedly calculate:

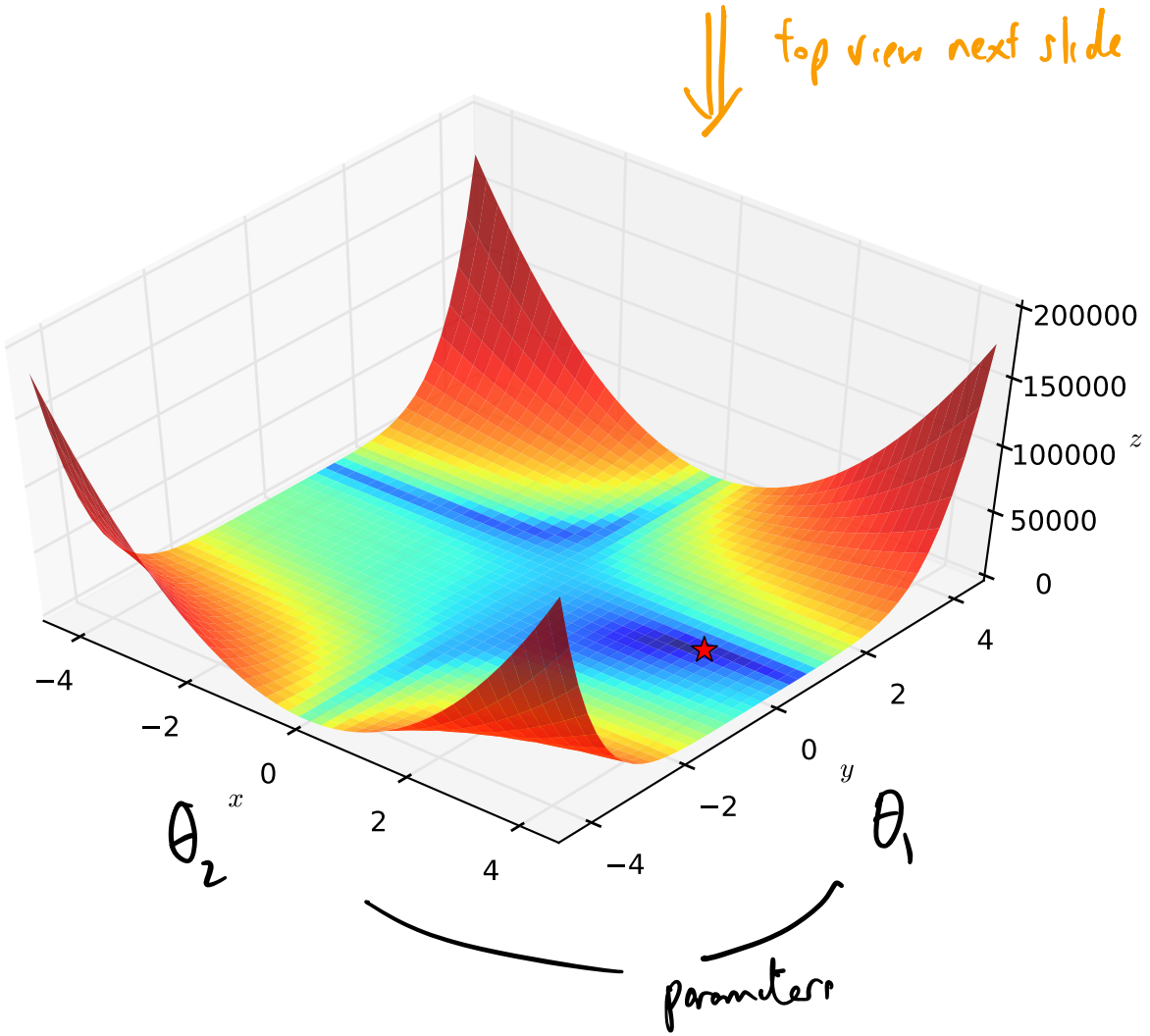
$$\mathbf{x} := \mathbf{x} - \eta \nabla_x f(\mathbf{x})$$

  step size
- η is typically called the *learning rate*. It can change over iterations. Setting the value of ϵ appropriately is an important part of deep learning.

Finding the optimal weights through gradient descent

Example:

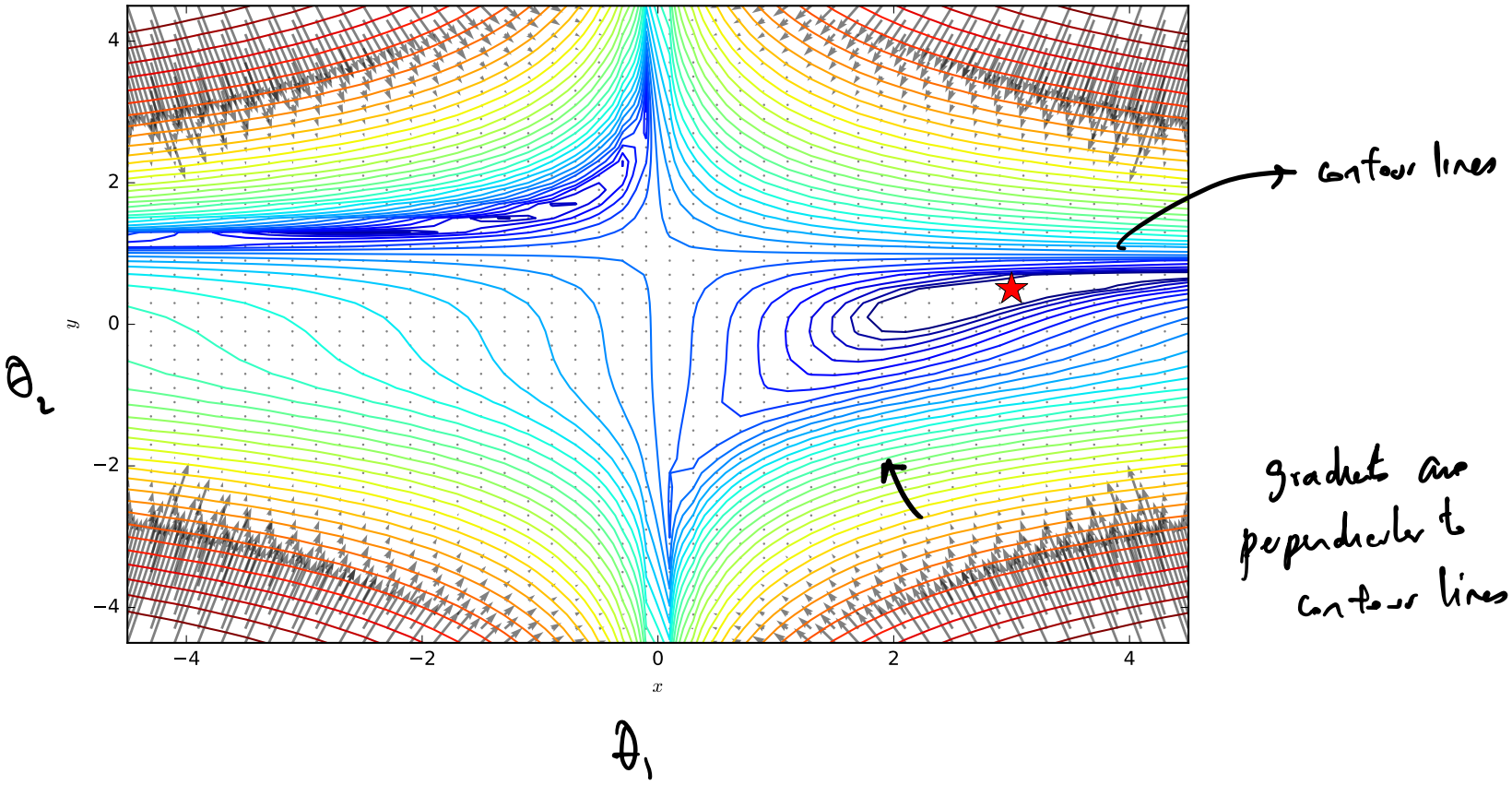
Animations thanks to: http://louistiao.me/notes/visualizing-and-animating-optimization-algorithms-with-matplotlib/



Finding the optimal weights through gradient descent

Example:

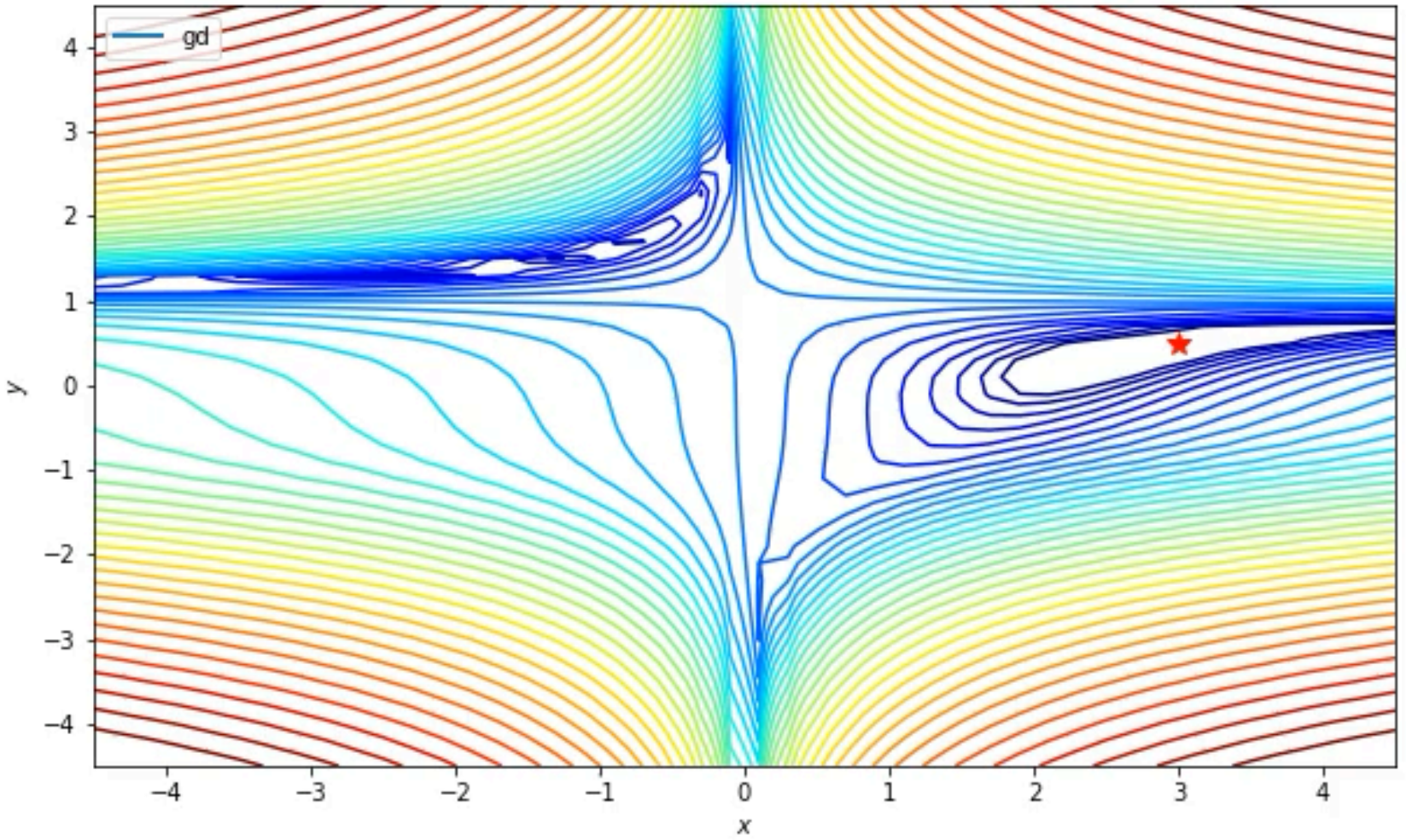
Animations thanks to: http://louistiao.me/notes/visualizing-and-animating-optimization-algorithms-with-matplotlib/



Finding the optimal weights through gradient descent

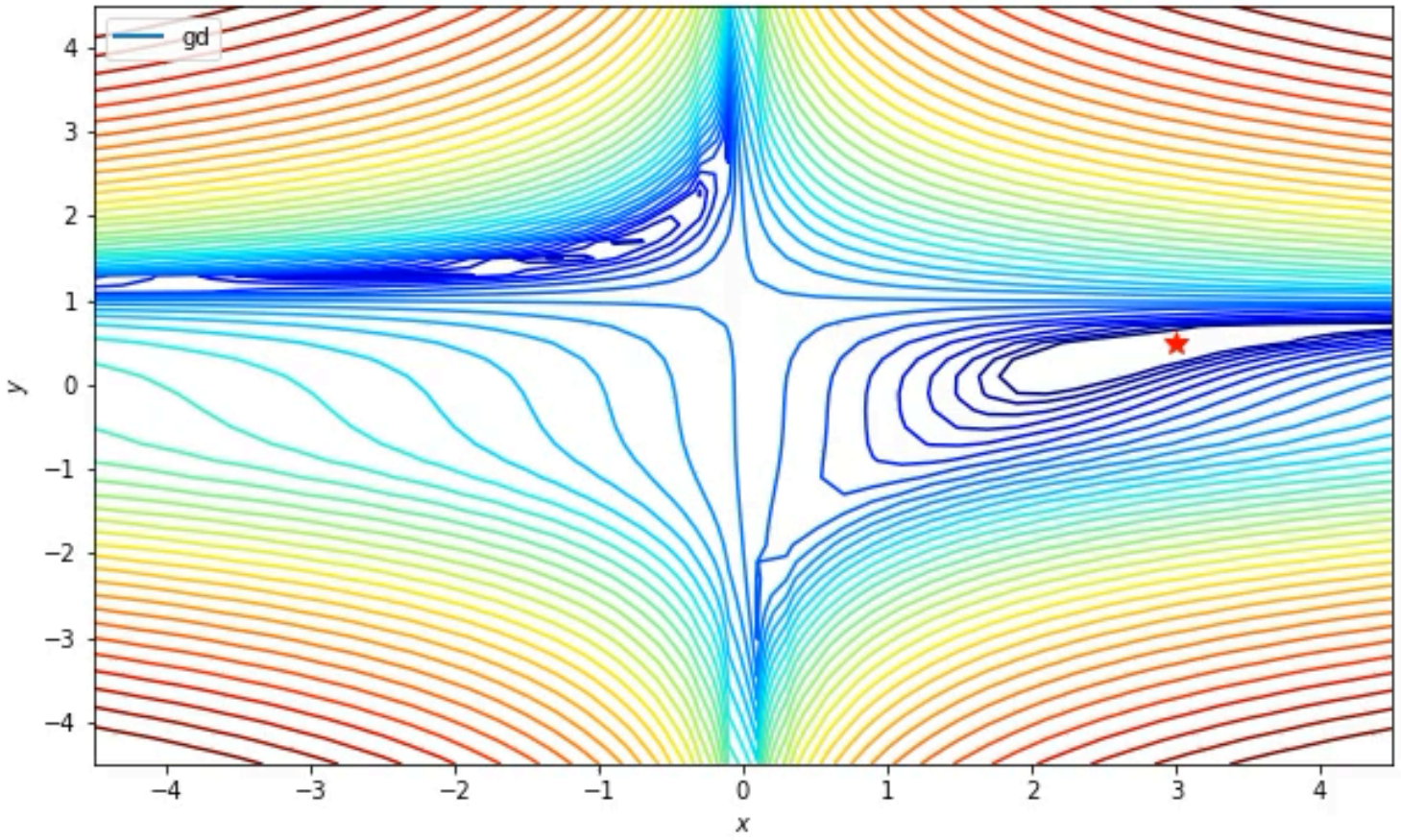
```
def gd(func, x0, eps=1e-4, tol=1e-3):
    last_diff = np.Inf
    x = x0
    path = [np.copy(x0)]
    costs = [func(x0)[0]]
    grads = []
    i = 1
    hit_max = False

    while last_diff > tol:
        cost, g = func(x) # returns the cost and the gradient
        x -= eps*g # gradient step
        last_diff = np.linalg.norm(x - path[-1]) # stopping criterion

        i += 1
        if i > max_iters:
            hit_max = True
            break
        path.append(np.copy(x))
        costs.append(cost)
        grads.append(g)

    return path, costs, grads, hit_max
```

Finding the optimal weights through gradient descent



Courtesy: Prof. Jonathan Kao

Finding the optimal weights through gradient descent



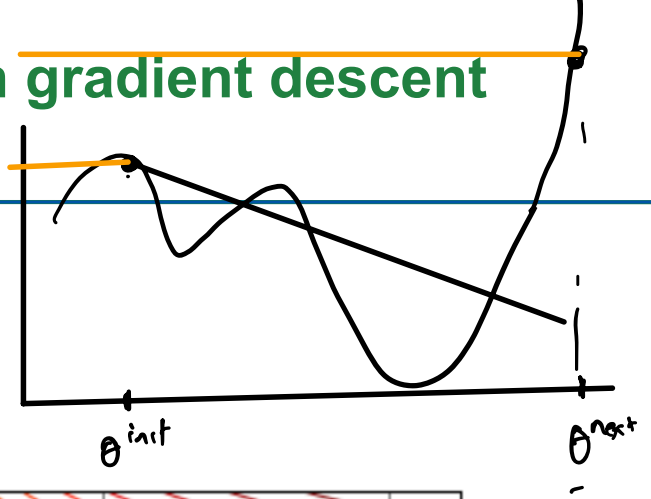
Courtesy: Prof. Jonathan Kao

Finding the optimal weights through gradient descent

$$\underline{\theta} \leftarrow \underline{\theta} - \eta \nabla_{\underline{\theta}} L(\underline{\theta})$$

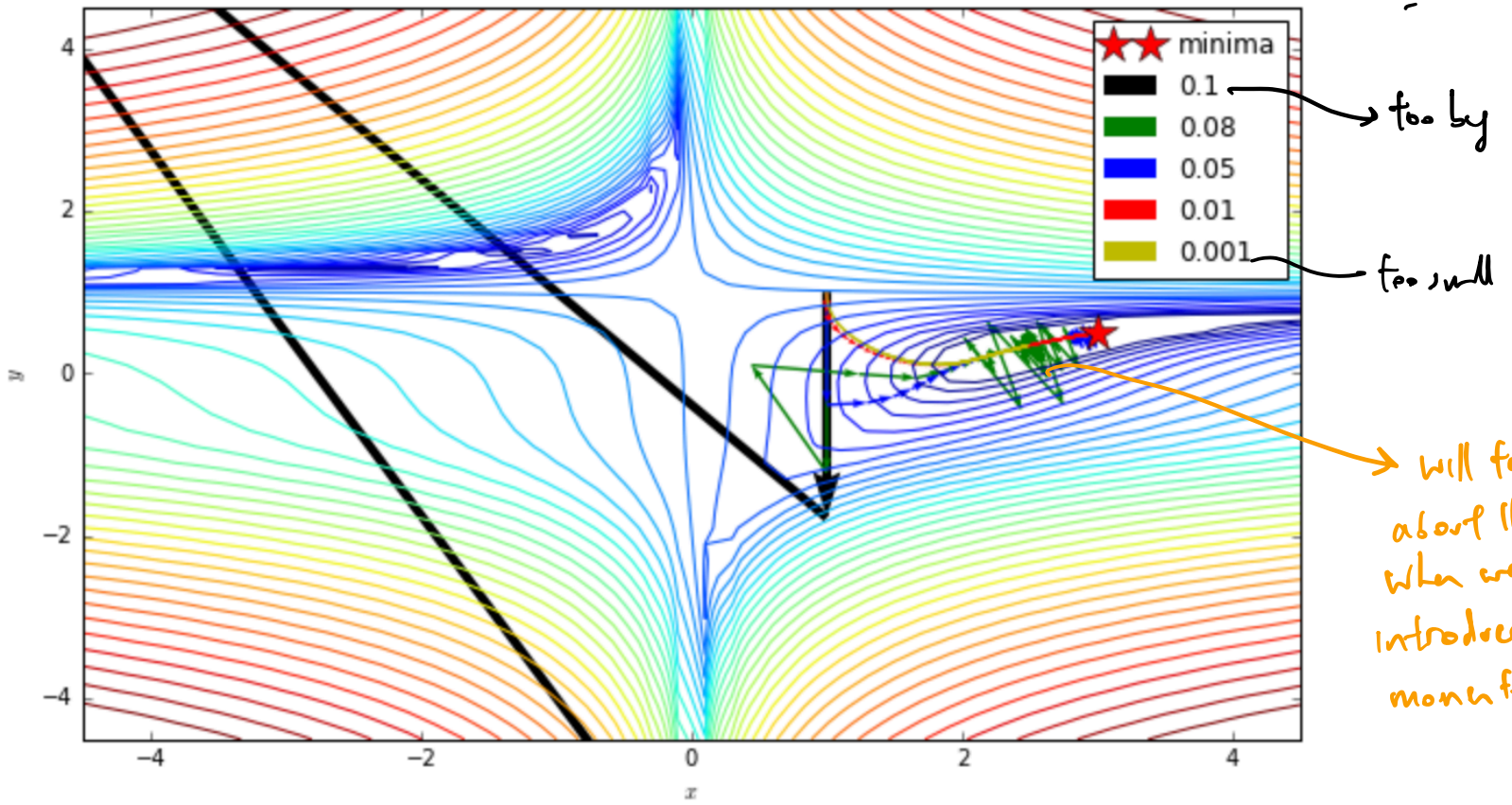
if too large it is not good

Taylor series expansion approx not valid



How do I pick the right step size?

If learning rate is too small then slow progress

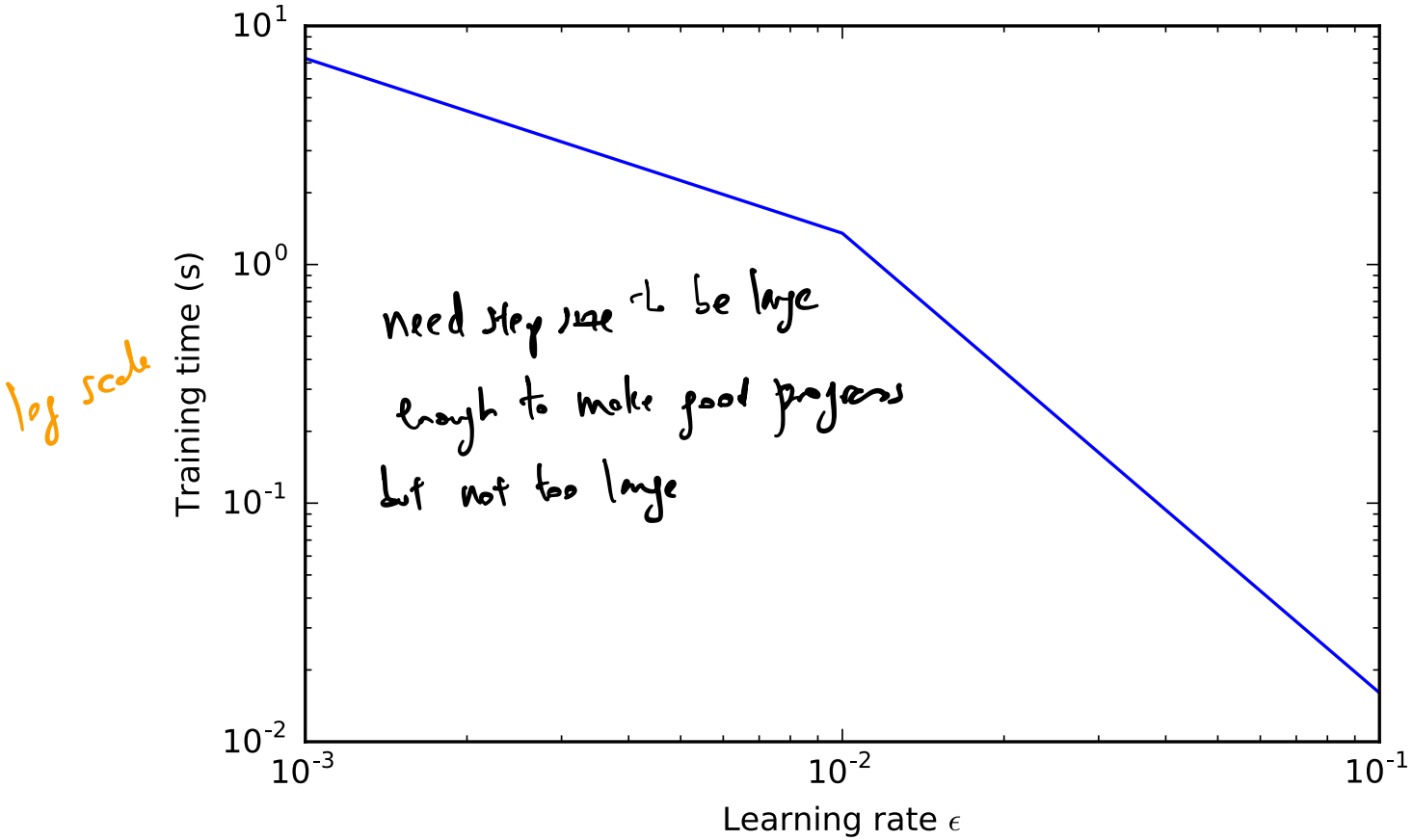


Finding the optimal weights through gradient descent

Why not always use smaller learning rates?

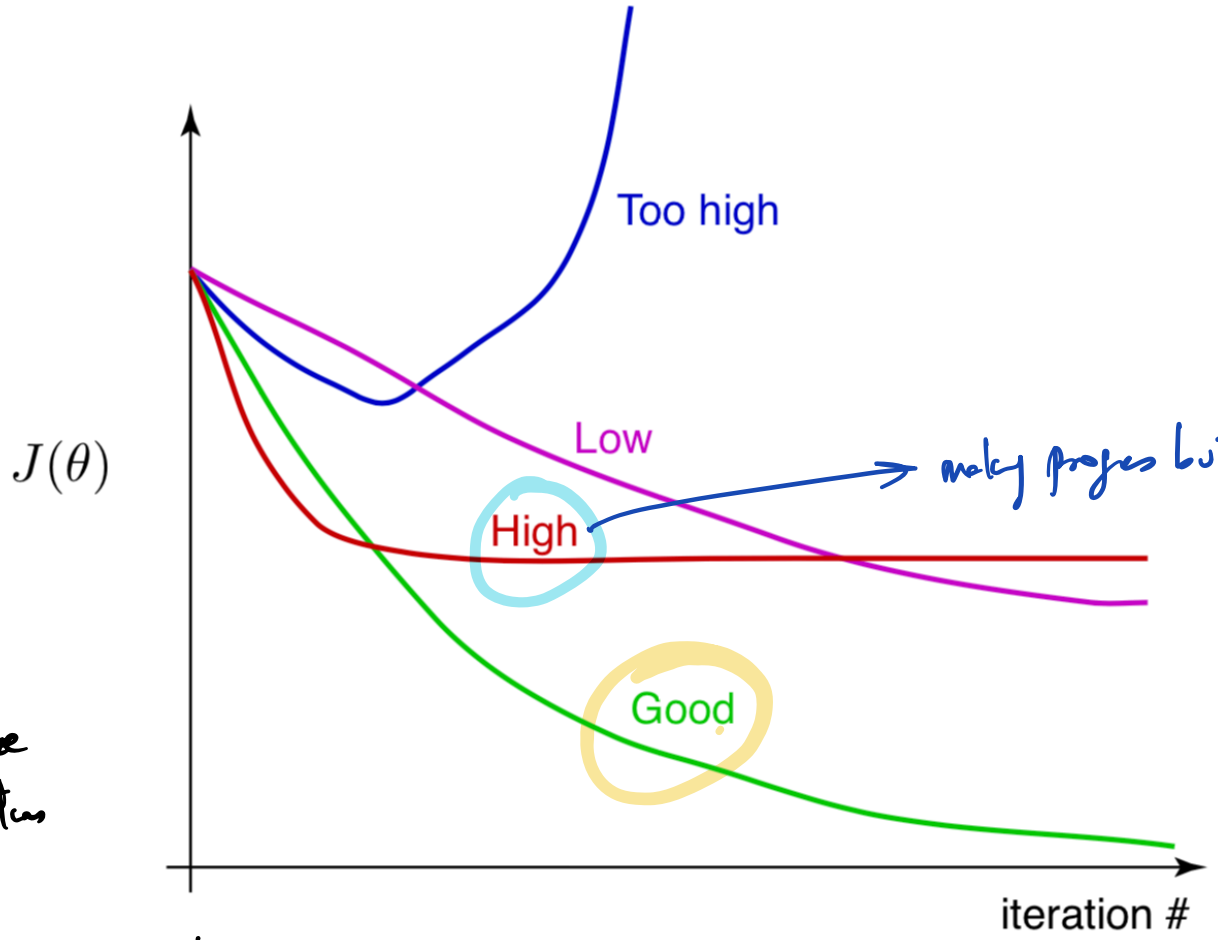
Line search → optimizes step size at each iteration

not used in high dim. optimization because it is costly



Interpreting the cost

The cost function can be very informative as to how to adjust your step sizes for gradient descent.



making progress but perhaps learning rate

learning rate schedule and annealing

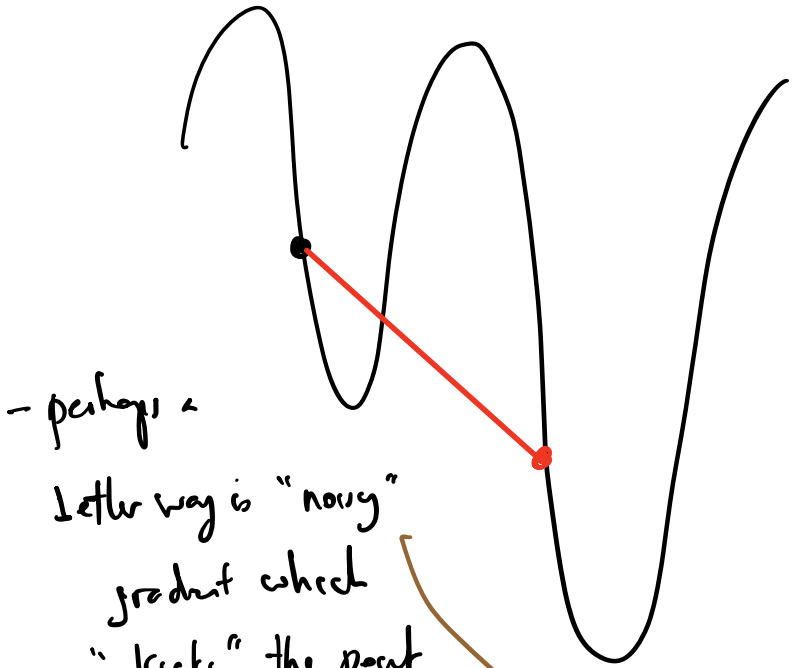
└ change step size appropriately as iterations proceed.

Start with large step size if loss plateau then reduce step size

Why not use a numerical gradient?

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

- Reasonable for softmax or small models
- too expensive for larger models
  - we have to evaluate $f(\cdot)$
  - almost like inference
  - needs memory



- perhaps another way is "noisy" gradient check "kicks" the point out of local minima

will do through stochastic gradient descent

Finding the optimal weights through gradient descent

How does this example differ from what we will really encounter?

- In this example, we know the function f() exactly, and thus at every point in space, we can calculate the gradient at that point exactly. (in the videos shown)
- In optimization, we differentiate the cost function f() with respect to the parameters.
  - The gradient of f() w.r.t. parameters is a function of the training data!
  - Hence, we can think of each data point as providing a noisy estimate of the gradient at that point.

$\nabla_{\underline{\theta}} \mathcal{L}(\underline{\theta})$ → softmax and MLE

$$\mathcal{L}(\underline{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \ell(\underline{x}^{(i)}, y^{(i)}) \approx \mathbb{E}_{\underline{x},y}\left[\ell(\underline{x},y)\right]$$

$$\nabla_{\underline{\theta}} \mathbb{E}_{(\underline{x},y)}\left[\ell(\underline{x},y)\right] = \mathbb{E}_{\underline{x},y}\left[\nabla_{\underline{\theta}}\, \ell(\underline{x},y)\right] \approx \frac{1}{N}\sum_{i=1}^{N} \nabla_{\underline{\theta}}\, \ell(\underline{x}^{(i)}, y^{(i)})$$

Finding the optimal weights through gradient descent

However, it's expensive to have to calculate the gradient by using *every example* in the training set.

To this end, we may want to get a noisier estimate of the gradient with fewer examples.

Instead of estimating the gradient $\Rightarrow \frac{1}{m}\sum_{i=1}^{m} \nabla_\theta l(x^{(i)}, y^{(i)})$

we use a subset of examples $\rightarrow$ stochastic gradient.

Batch vs minibatch (cont)

Calculating the gradient exactly is expensive, because it requires evaluating the model on all m examples in the dataset. This leads to an important distinction.

- Batch algorithm: uses all m examples in the training set to calculate the gradient. → full gradient
- Minibatch algorithm: approximates the gradient by calculating it using k training examples, where $m > k > 1$. Use $k = 32, 64, 128$ etc
- Stochastic algorithm: approximates the gradient by calculating it over one example. Stochastic GD $\rightarrow$ $k = 1$

random subset of k data samples. Samples k out of m examples and calculates $\frac{1}{k}\sum_{i=1}^{k} \nabla_\theta l(x^{(i)}, y^{(i)})$

"noisy" gradient

It is typical in deep learning to use minibatch gradient descent. Note that some may also use minibatch and stochastic gradient descent interchangeably.

Theory: Shows that SGD has an implicit regularization

Finding the optimal weights through gradient descent

You'll do this in the HW. More on this later in the optimization lecture…

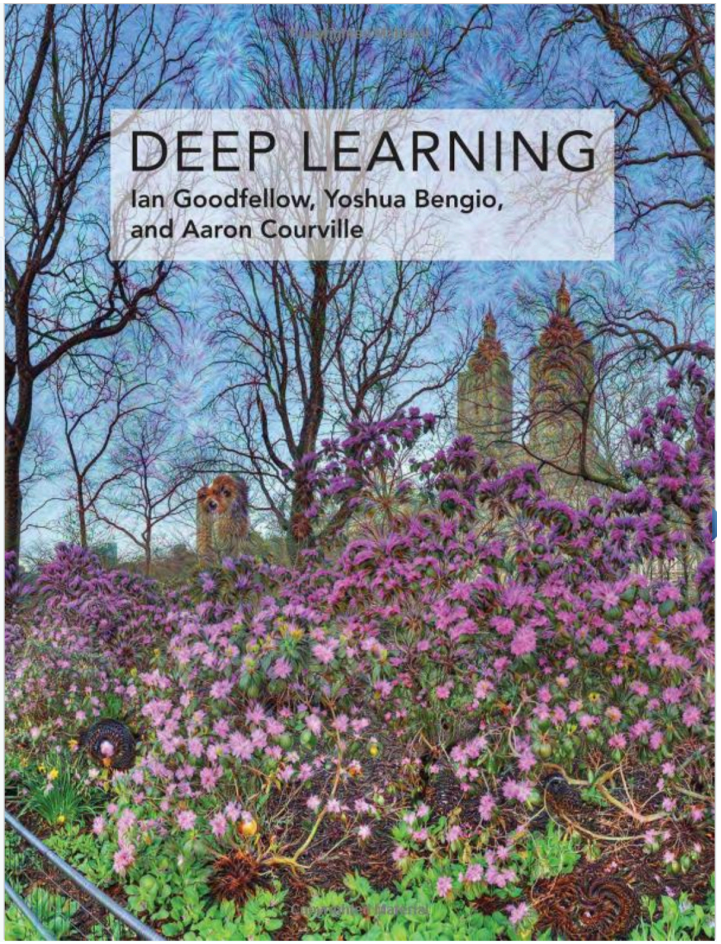
- And a lot more to be said about optimization.
- First order vs second order methods
- Momentum
- Adaptive gradients.
- … all of these will become quite important when we get to neural networks. We will cover these in an optimization lecture.

Neural networks

We will introduce the neural network architecture, parameters, and its inspiration from biological neurons.

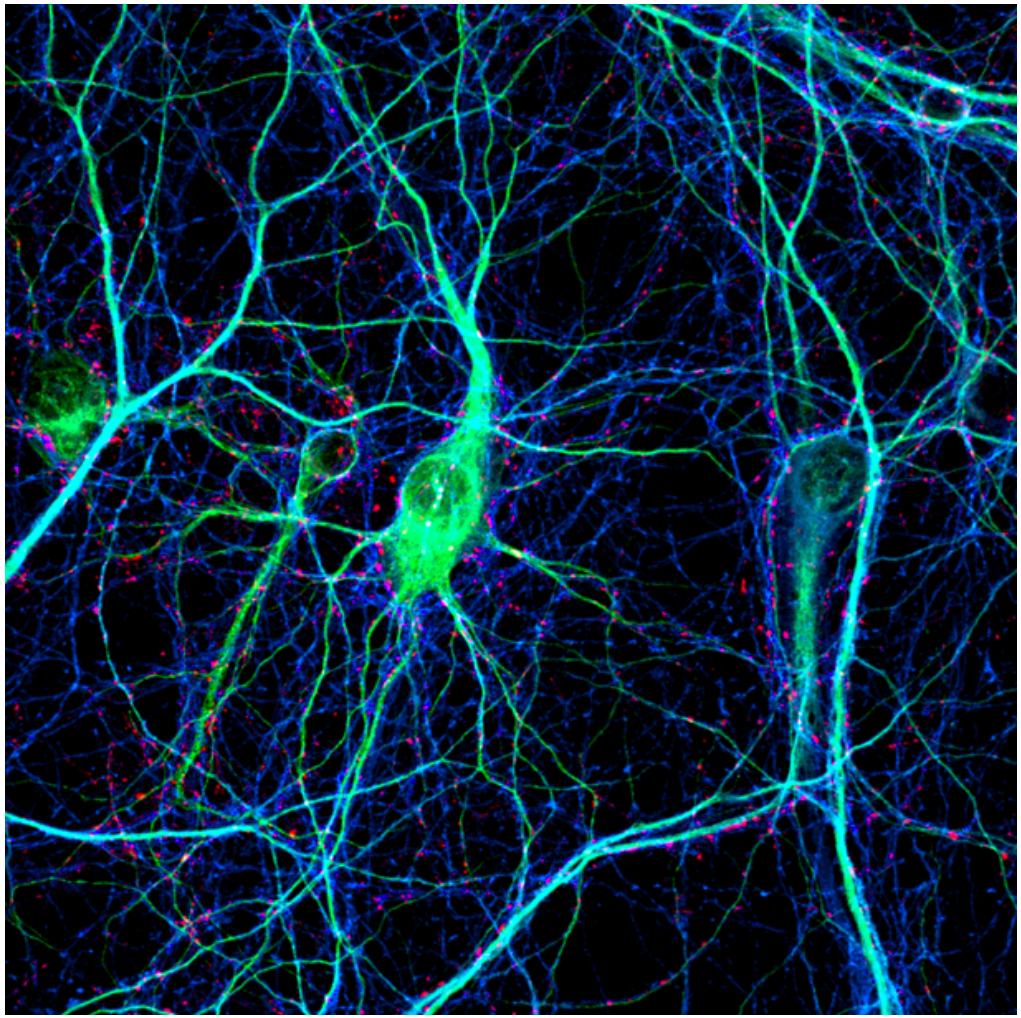
Reading:

Deep Learning, 6 (intro), 6.1, 6.2, 6.3, 6.4

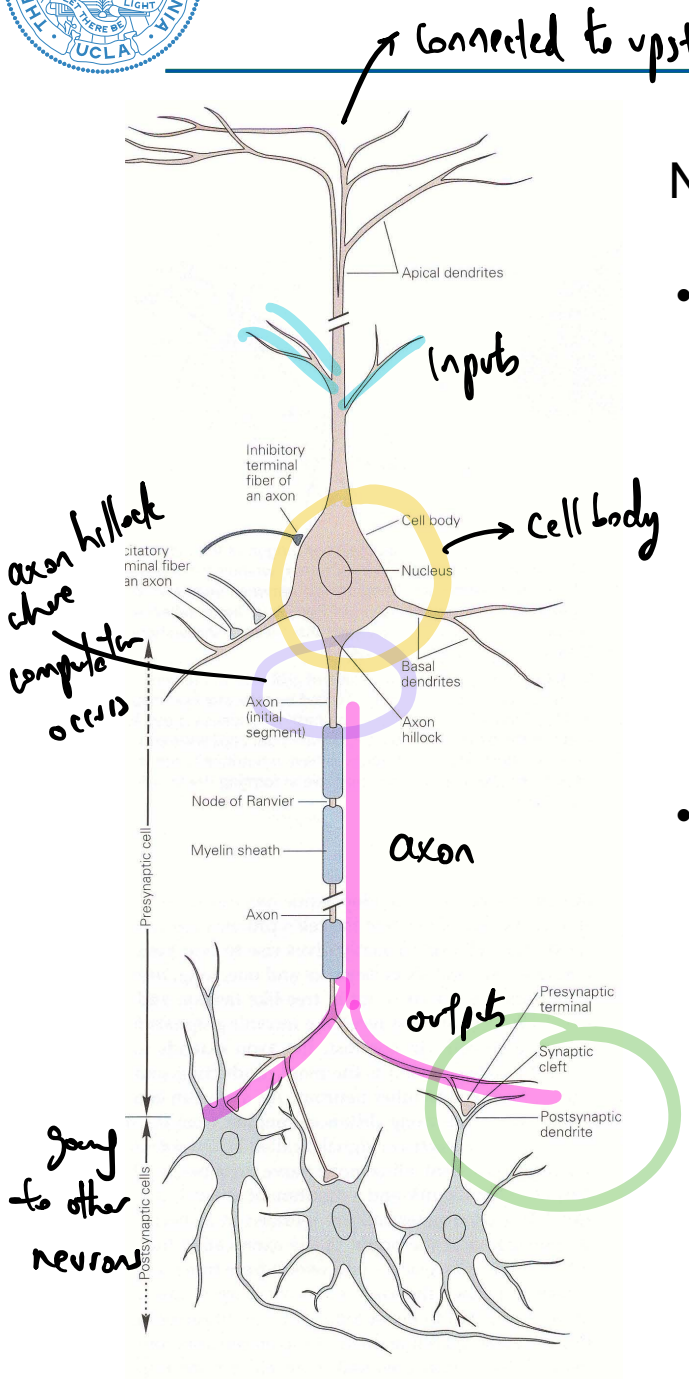


Inspiration from neuroscience

Biological neurons → McCollogh & Pitts 1943



Inspiration from neuroscience



Neurons are the main signaling units of the nervous system.

- Neurons have four regions:
  1) Cell body (soma) – metabolic center, with nucleus, etc.
  2) Dendrites – tree like structure for receiving **input** signals.
  3) Axon – single, long, tubular structure for sending **output** signals.
  4) Presynaptic terminals – sites of communication to next neurons.

- Axons (the **output)** convey signals to other neurons:
  - Conveys electrical signals long distances (0.1mm – 3 m).
  - Conveys action potentials (~100 mV, ~1 ms pulses).
  - Action potentials initiate at the axon hillock.
  - Propagate w/o distortion or failure at 1-100 m/s.

$$f(z_1, z_2, z_3) = \begin{cases} 1 & \text{if } z_1 + z_2 + z_3 > T \\ 0 & \text{else} \end{cases}$$

fires beyond a threshold

Inspiration from neuroscience

Neurons are diverse (unlike in neural networks)

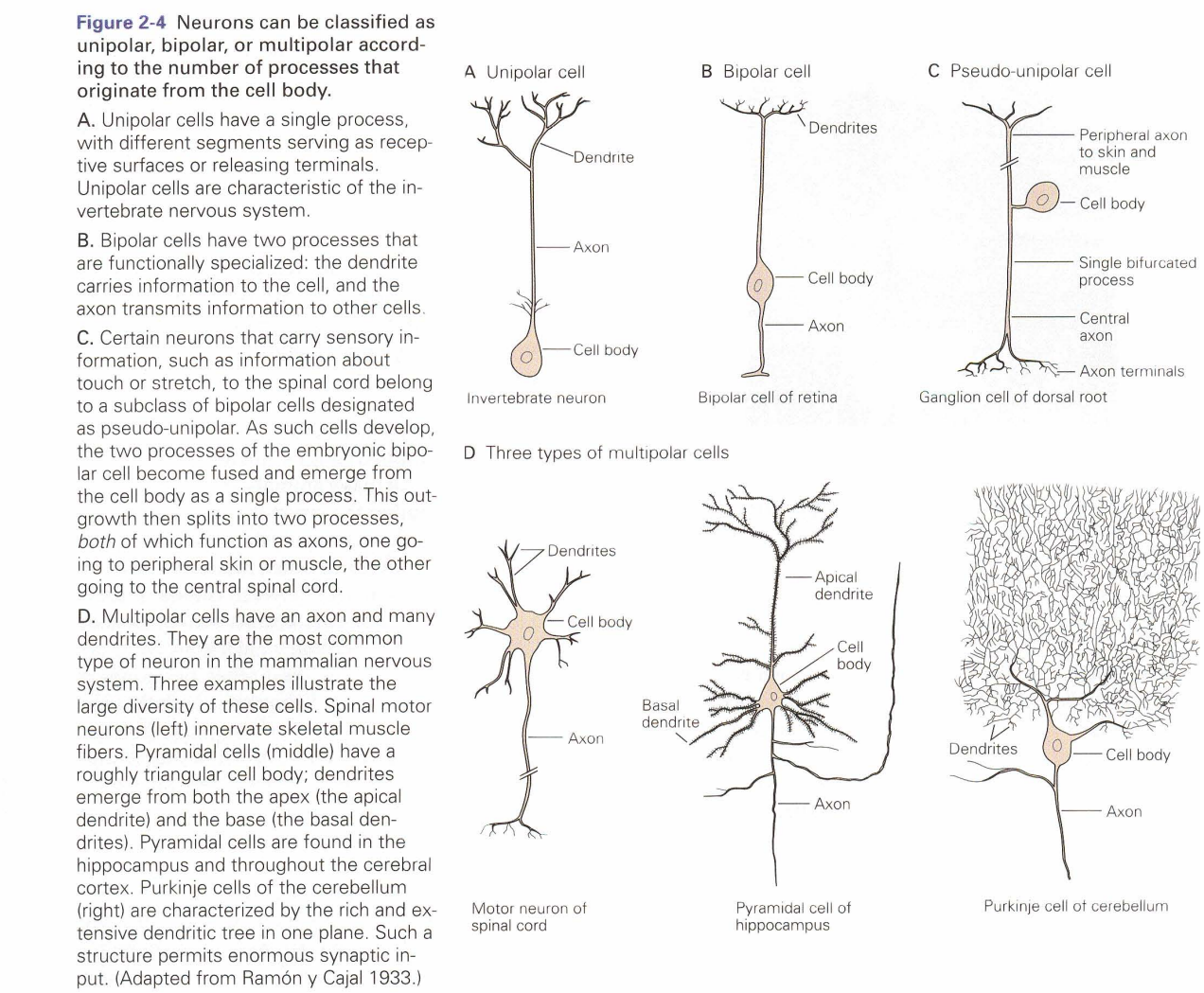


Figure 2-4 Neurons can be classified as unipolar, bipolar, or multipolar according to the number of processes that originate from the cell body.

A. Unipolar cells have a single process, with different segments serving as receptive surfaces or releasing terminals. Unipolar cells are characteristic of the invertebrate nervous system.

B. Bipolar cells have two processes that are functionally specialized: the dendrite carries information to the cell, and the axon transmits information to other cells.

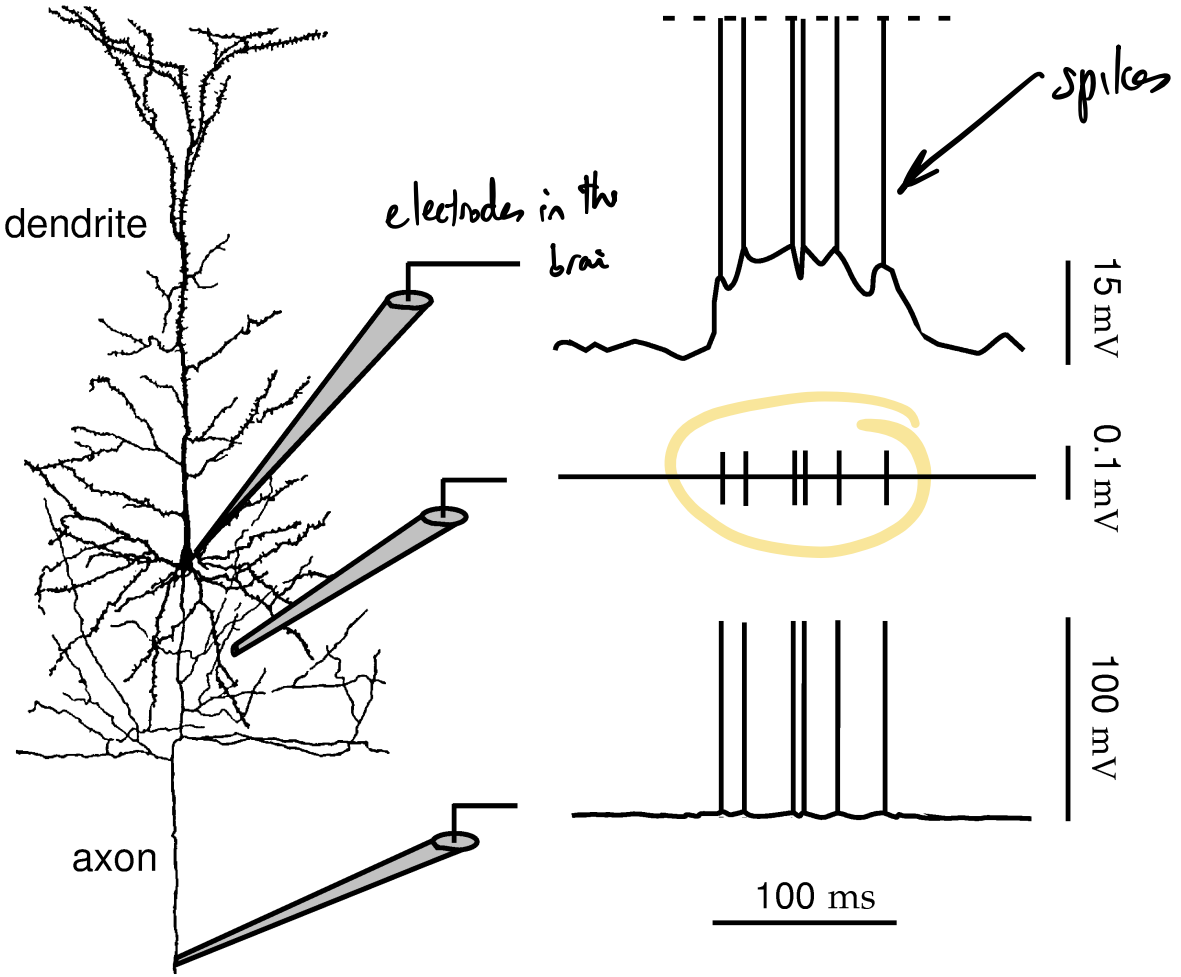
C. Certain neurons that carry sensory information, such as information about touch or stretch, to the spinal cord belong to a subclass of bipolar cells designated as pseudo-unipolar. As such cells develop, the two processes of the embryonic bipolar cell become fused and emerge from the cell body as a single process. This outgrowth then splits into two processes, *both* of which function as axons, one going to peripheral skin or muscle, the other going to the central spinal cord.

D. Multipolar cells have an axon and many dendrites. They are the most common type of neuron in the mammalian nervous system. Three examples illustrate the large diversity of these cells. Spinal motor neurons (left) innervate skeletal muscle fibers. Pyramidal cells (middle) have a roughly triangular cell body; dendrites emerge from both the apex (the apical dendrite) and the base (the basal dendrites). Pyramidal cells are found in the hippocampus and throughout the cerebral cortex. Purkinje cells of the cerebellum (right) are characterized by the rich and extensive dendritic tree in one plane. Such a structure permits enormous synaptic input. (Adapted from Ramón y Cajal 1933.)

many different neuron types, size, function etc

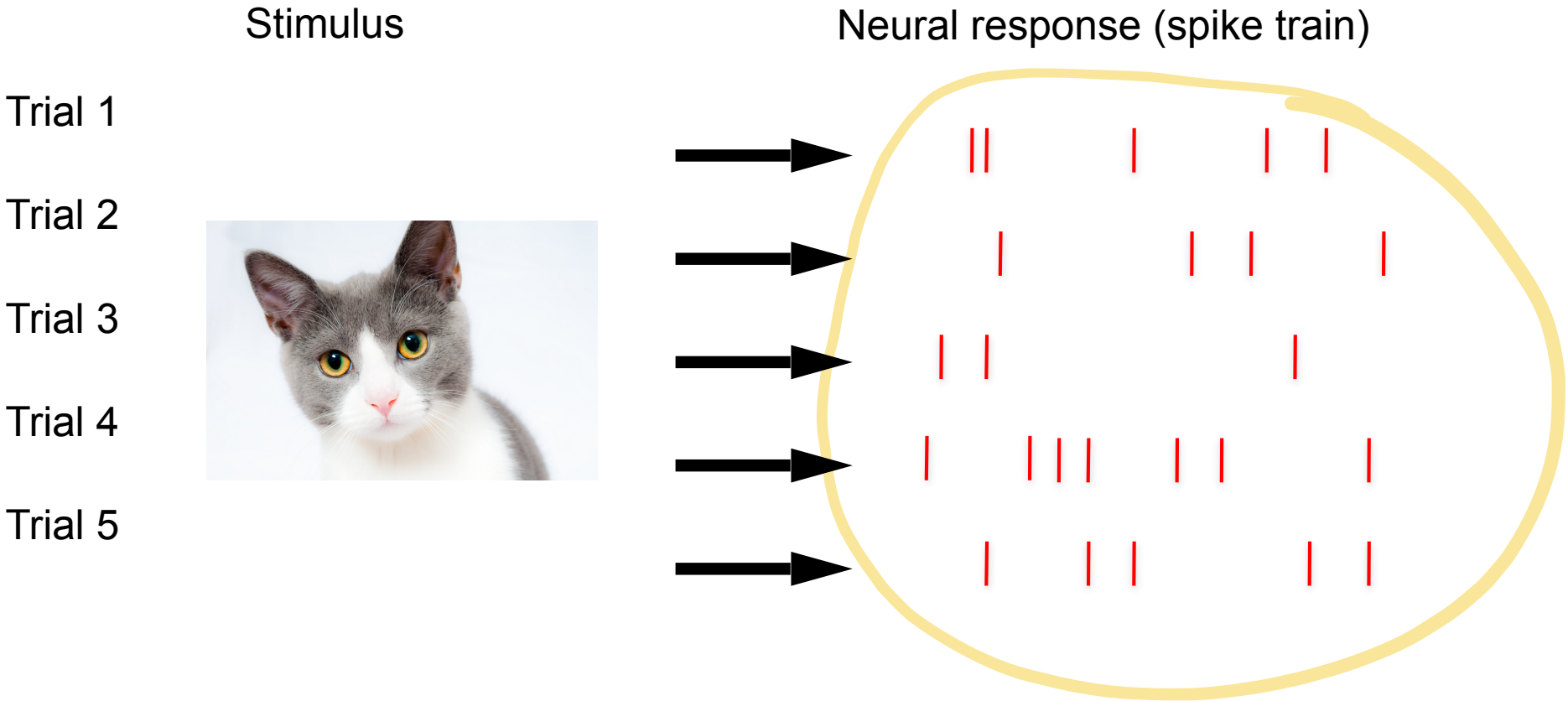
Inspiration from neuroscience

Neurons fundamentally communicate through all-or-nothing spikes (not analog values!):



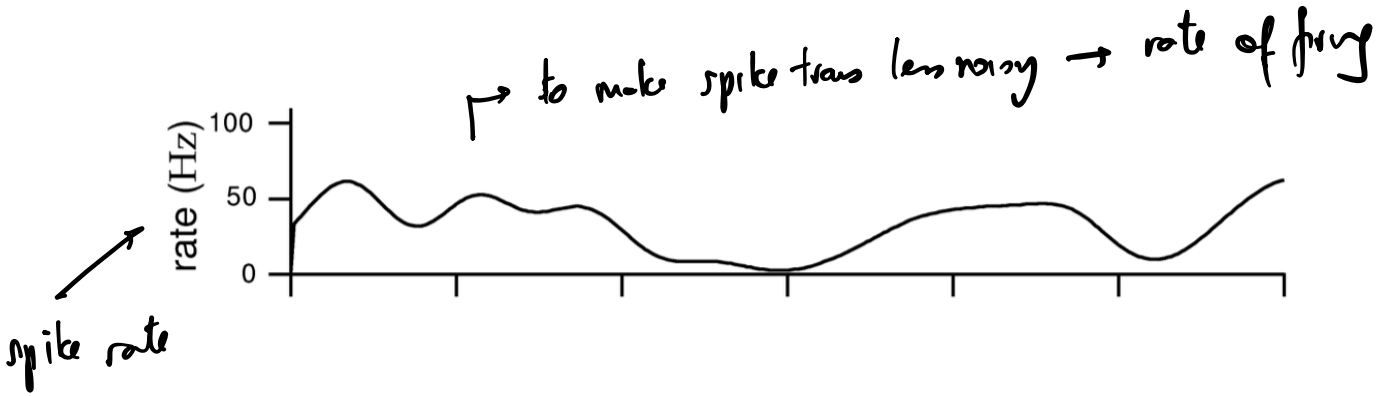
Inspiration from neuroscience

… And the spikes are probabilistic.



Inspiration from neuroscience

The spikes reflect an underlying rate.



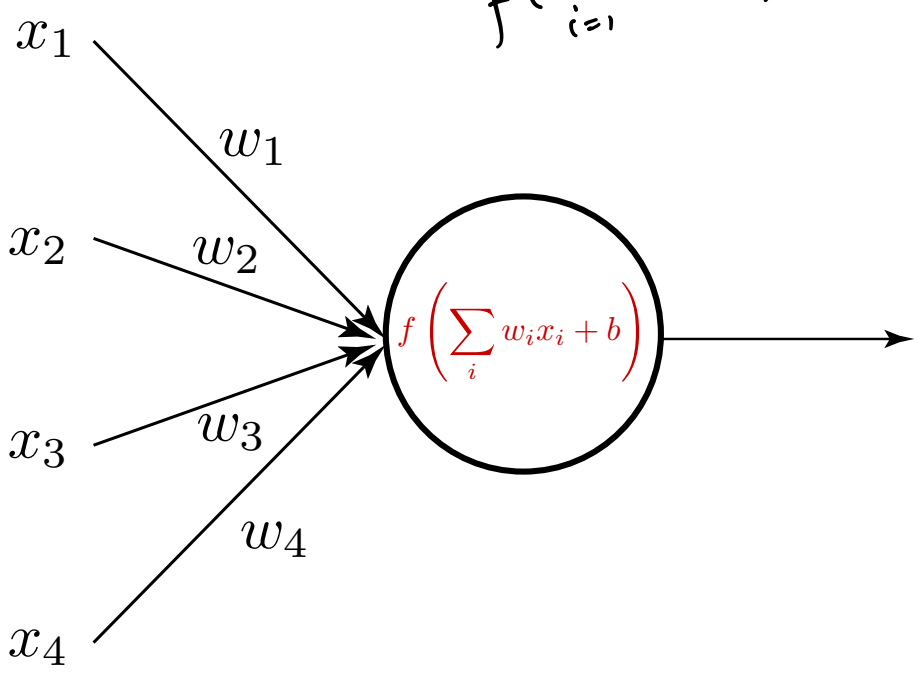
This rate is what the neural networks are “encoding.”

Inspiration from neuroscience

How does the artificial neuron compare to the real neuron?

↳ ~ 1940s / 1950s

$f\left(\sum_{i=1}^{n} w_i x_i\right)$



Inspiration from neuroscience

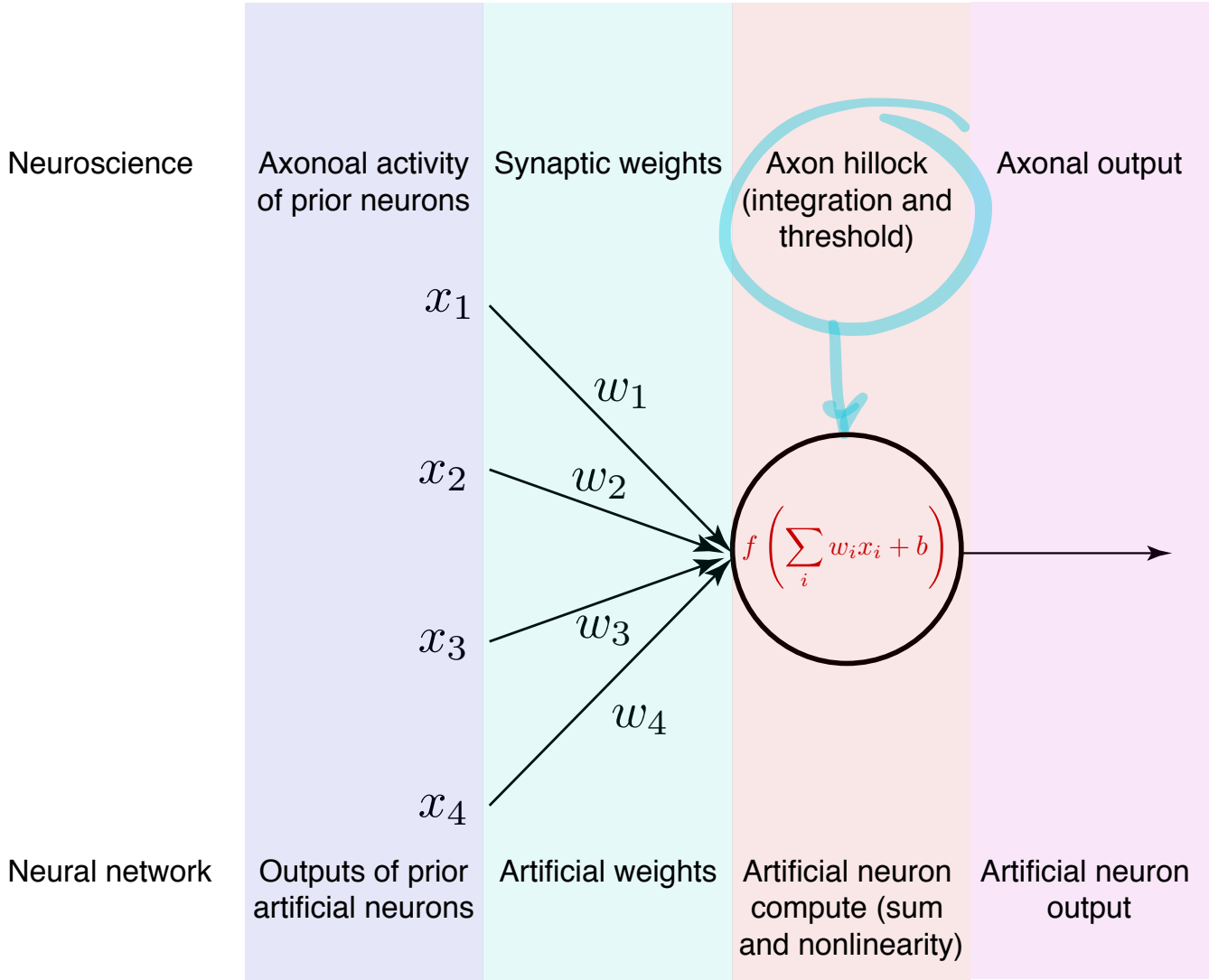
How does the artificial neuron compare to the real neuron?

The artificial neuron (cont.)

- The incoming signals, a vector $\mathbf{x} \in \mathbb{R}^N$, reflects the output of N neurons that are connected to the current artificial neuron.
- The incoming signals, $\mathbf{x}$, are pointwise multiplied by a vector, $\mathbf{w} \in \mathbb{R}^N$. That is, we calculate $w_i x_i$ for $i = 1, \ldots, N$. This computation reflects dendritic processing.
- The "dendritic-processed" signals are then summed, i.e., we calculate $\sum_i w_i x_i + b$. This computation reflects integration at the axon hillock (the first "Node of Ranvier") where action potentials are generated if the integrated signal is large enough.
- The output of the artificial neuron is then a nonlinearly transformation of the integrated signal, i.e., $f(\sum_i w_i x_i + b)$. Rather than reflecting whether an action potential was generated or not (which is a noisy process), this nonlinear output is typically treated as the *rate* of the neuron. The higher the rate, the more likely the neuron is to fire action potentials.

Inspiration from neuroscience

How does the artificial neuron compare to the real neuron?



Caution when comparing to biology

These computing analogies are not precise, with large approximations. Limitations in the analogy include:

- Synaptic transmission is probabilistic, nonlinear, and dynamic.
- Dendritic integration is probabilistic and may be nonlinear.
- Dendritic computation has associated spatiotemporal decay.
- Integration is subject to biological constraints; for example, ion channels (which change the voltage of the cell) undergo refractory periods when they do not open until hyperpolarization.
- Different neurons may have different action potential thresholds depending on the density of sodium-gated ion channels.
- Feedforward and convolutional neural networks have no recurrent connections.
- Many different cell types.
- Neurons have specific dynamics that can be modulated by e.g., calcium concentration.
- And so many more...

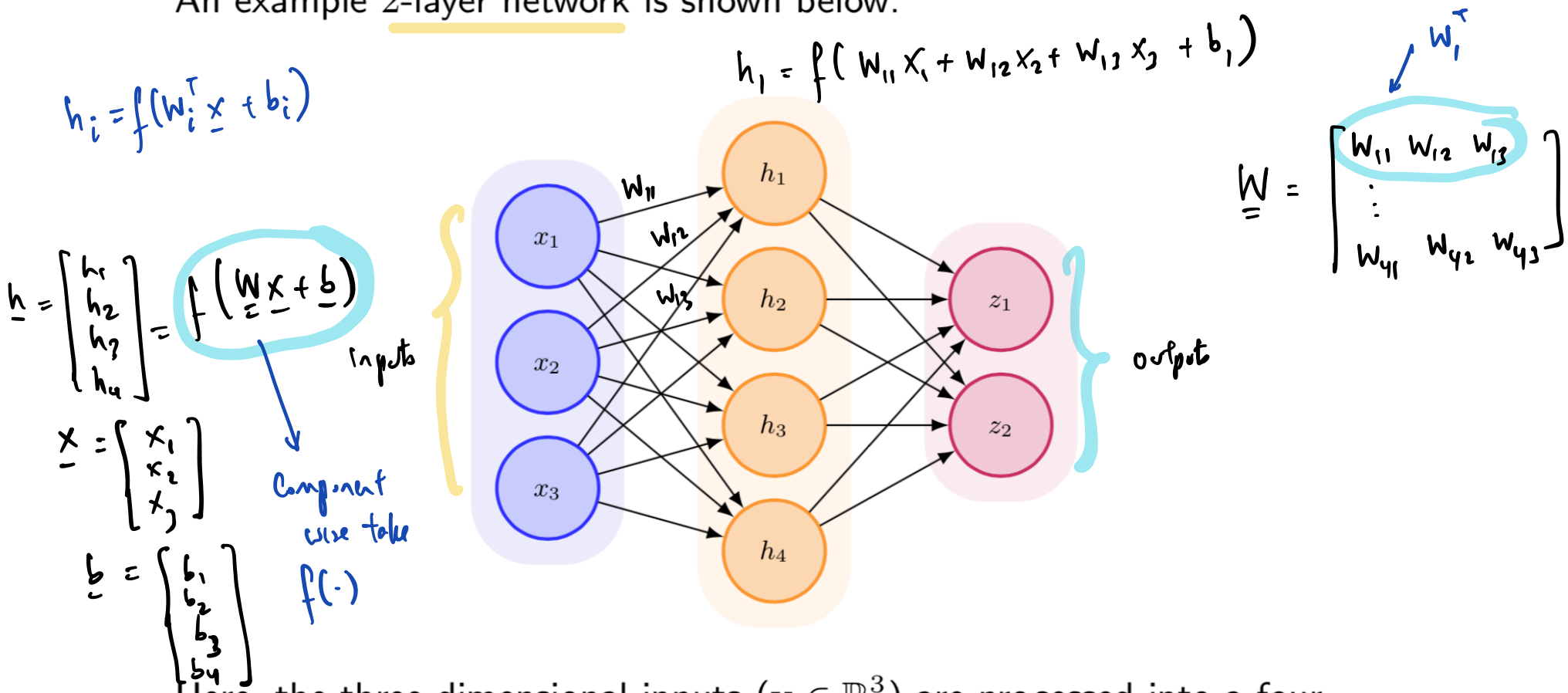
Nomenclature

Some naming conventions.

- We call the first layer of a neural network the "input layer." We typically represent this with the variable $\mathbf{x}$.
- We call the last layer the "output layer." We typically represent this with the variable $\mathbf{z}$. (Note: why not $\mathbf{y}$ to match our prior nomenclature for the supervised outputs? Because the output of the network may be a processed version of $\mathbf{z}$, e.g., $\mathrm{softmax}(\mathbf{z})$.)
- We call the intermediate layers the "hidden layers." We typically represent this with the variable $\mathbf{h}$.
- When we specify that a network has N layers, this does not include the input layer.

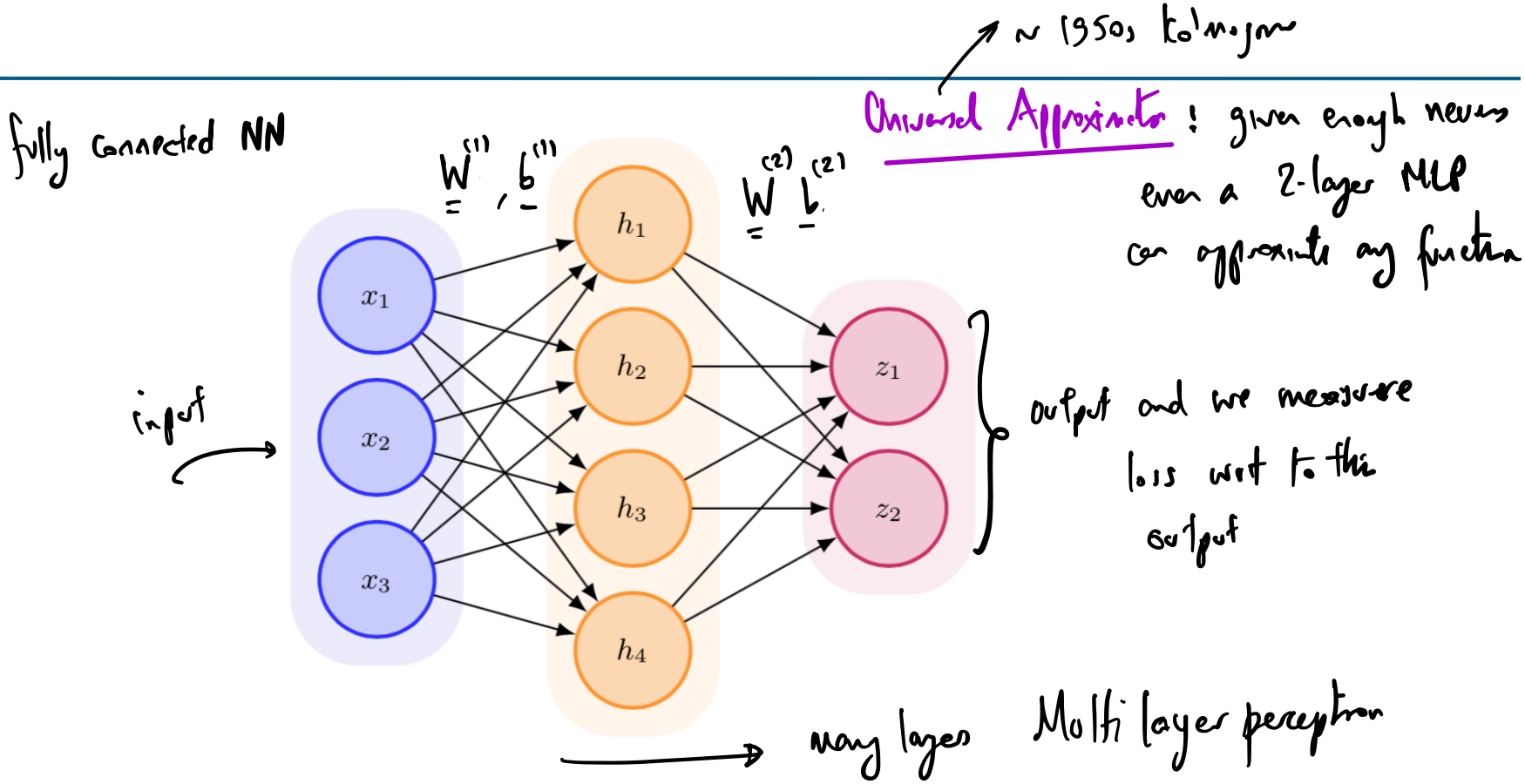
Neural network architecture

An example 2-layer network is shown below.



Here, the three dimensional inputs ($\mathbf{x} \in \mathbb{R}^3$) are processed into a four dimensional intermediate representation ($\mathbf{h} \in \mathbb{R}^4$), which are then transormed into the two dimensional outputs ($\mathbf{z} \in \mathbb{R}^2$).

Neural networks



First layer:

$$\mathbf{h} = f\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right)$$

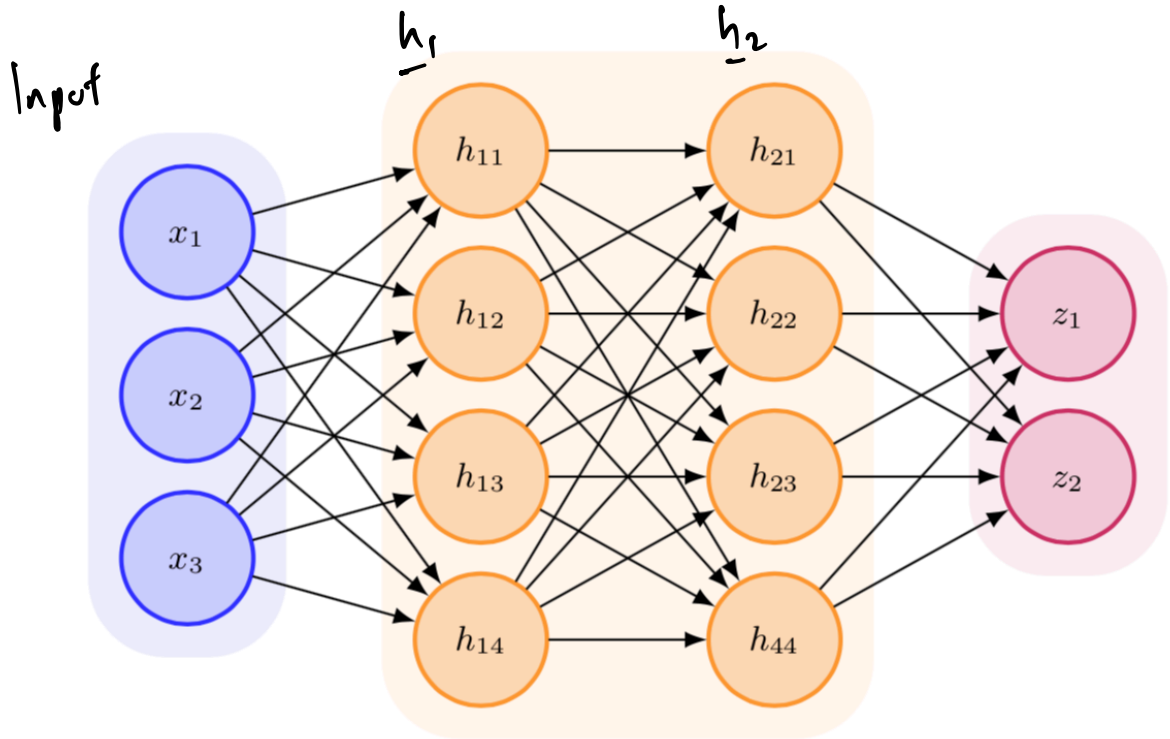
Second (output layer):

$$\mathbf{z} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$

This network has 6 neurons (not counting the input). It has (3x4) + (4x2) = 20 weights, and 4+2 = 6 biases for a total of 26 learnable parameters.

Neural network architecture 2

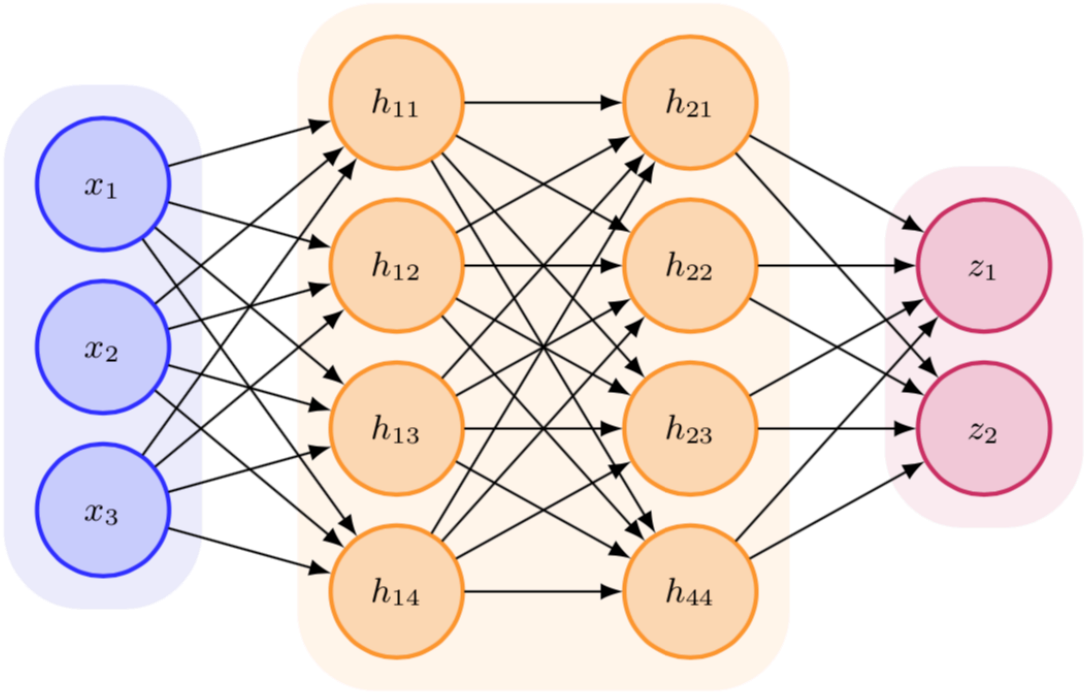
An example 3-layer network is shown below.



Can add more layers
↓ depth = # of layers

Here, h_{ij} denotes the jth element of $\mathbf{h}_i$. There are many considerations in architecture design, which we will later discuss.

Neural networks

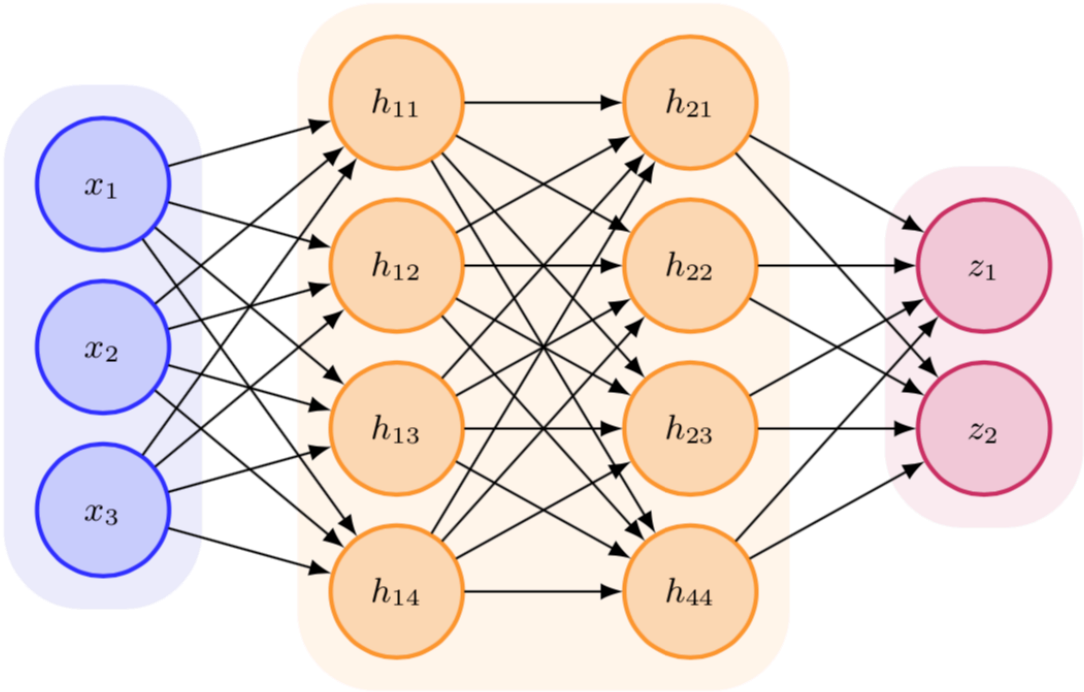


First layer: $\mathbf{h}_1 = f(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$

Second layer: $\mathbf{h}_2 = f(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$

Third (output layer): $\mathbf{z} = \mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3$

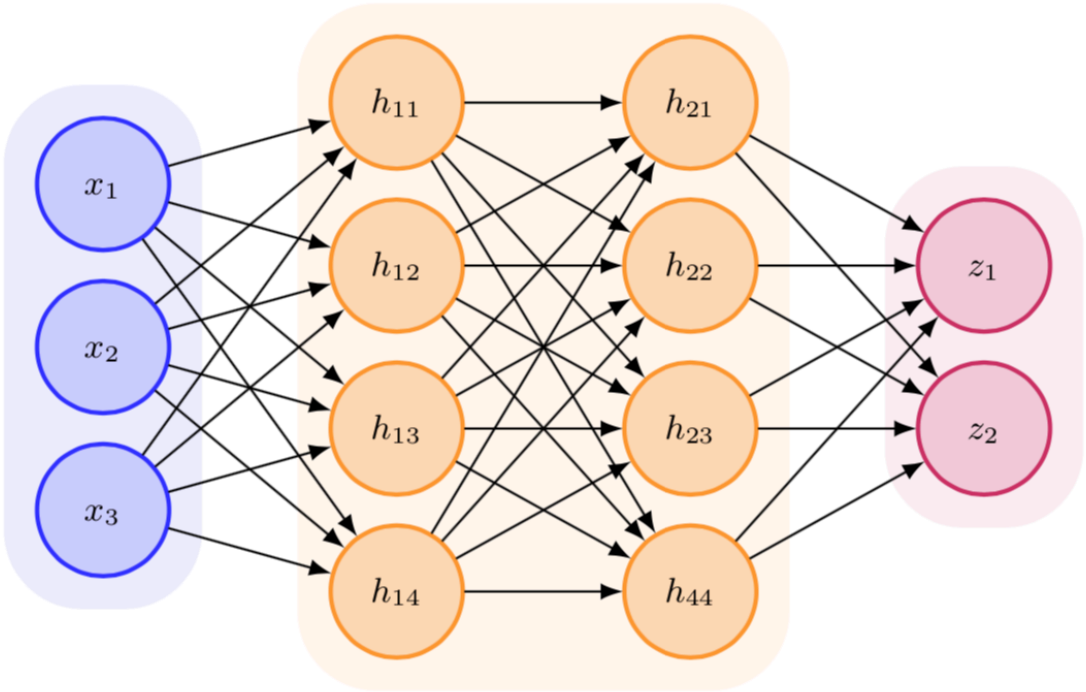
Neural networks



This network has 10 neurons (not counting the input). It has (3x4) + (4x4) + (4x2) = 36 weights, and 4+4+2= 10 biases for a total of 46 learnable parameters.

Perspective: Convolutional neural networks typically have on the order of hundreds of millions of parameters.

Neural networks



$f(\cdot)$

```
# Define the activation function
f = lambda x: x * (x > 0)
# Forward pass of a 3-layer network
h1 = f(np.dot(W1, x) + b1)
h2 = f(np.dot(W2, h1) + b2)
z =   np.dot(W3, h2) + b3
```

→ ReLU function (will see soon)

What if f() is linear?

Neural networks

even identity function $f(x) = x$

The above figure suggests the following equation for a neural network.

- Layer 1: $\mathbf{h}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$
- Layer 2: $\mathbf{h}_2 = \mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2$
- $\vdots$
- Layer N: $\mathbf{z} = \mathbf{W}_N\mathbf{h}_{N-1} + \mathbf{b}_N$

$$\underline{h}_2 = \underline{\underline{W}}_2 \underline{h}_1 + \underline{b}_2 = \underline{\underline{W}}_2 \left[\underline{\underline{W}}_1 \underline{x} + \underline{b}_1\right] + \underline{b}_2$$

$$\underline{h}_2 = \underline{\tilde{W}}_2 \underline{x} + \underline{\tilde{b}}_2 \begin{cases} = \underline{\underline{W}}_2 \underline{\underline{W}}_1 \underline{x} + \underline{\underline{W}}_2 \underline{b}_1 + \underline{b}_2 \\ \underline{\tilde{W}}_2 \qquad\qquad \underline{\tilde{b}}_2 \end{cases}$$

Any composition of linear functions can be reduced to a single linear function.
Here, $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, where

$$\mathbf{W} = \mathbf{W}_N \cdots \mathbf{W}_2\mathbf{W}_1$$

and

$$\mathbf{b} = \mathbf{b}_N + \mathbf{W}_N\mathbf{b}_{N-1} + \cdots + \mathbf{W}_N \cdots \mathbf{W}_3\mathbf{b}_2 + \mathbf{W}_N \cdots \mathbf{W}_2\mathbf{b}_1$$

What if f() is linear?

- This may be useful in some contexts. For example, when $\dim(\mathbf{h}) \ll \dim(\mathbf{x})$, this corresponds to finding a low-rank representation of the inputs.
- However, a system with greater complexity may require a higher capacity model.